

Module 2: Lesson 1

Convolutional Neural Networks: Introduction



Outline

- ▶ From fully connected layers to convolutions
- ▶ Convolutions for images
- ▶ Padding and stride

Convolutional neural network

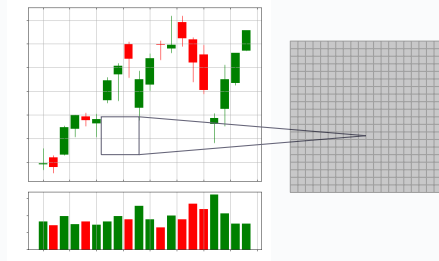
Image data presents a structured source of information that differs from the one-dimensional vectors that we analyze in the context of MLPs.

An image can be represented as a two-dimensional grid of pixels of dimension height $\times$ width.

We can use images as inputs of MLPs by flattening those pixels.

However, we can preserve and exploit the spatial structure of each image's pixels to obtain better models for image recognition.

Convolutional Neural Networks (CNNs) represent a powerful family of neural networks designed for this purpose.



From fully connected layers to convolutions

Consider an MLP that uses two-dimensional images X as inputs, with H representing the immediate hidden representation with the same shape as X .

A fully-connected layer that maps X to H has, at each location (i, j) in H , the following expression:

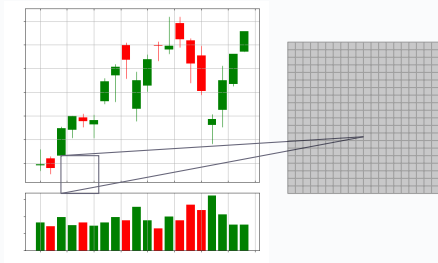
$$[H]_{i,j} = [U]_{i,j} + \sum_a \sum_b [V]_{i,j,a,b} [X]_{i+a,j+b} \quad (1)$$

The hidden representation $[H]_{i,j}$ comes from a bias term $[U]_{i,j}$ plus a summation over all the image's pixels, each weighted by $[V]_{i,j,a,b}$, the parameter of pixel's $(i+a, j+b)$ of input $[X]_{i+a,j+b}$ (V is a fourth-order tensor).

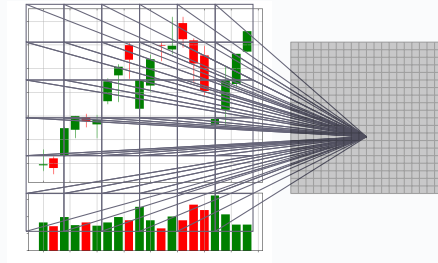
Notice that for each hidden state $[H]_{i,j}$ we would have specific weights that would link pixel (i, j) to all the other pixels in the image. That is, in a 1-megapixel image we would face 10^{12} parameters to estimate (aside from the biases).

Convolutional neural network

Hidden unit, $[H]$



Hidden unit, $[H]$



From fully connected layers to convolutions

To make the problem computationally feasible, we first impose that the biases and weights are the same across pixels that are at the same distance from each hidden representation:

$$[H]_{i,j} = u + \sum_a \sum_b [V]_{a,b} [X]_{i+a,j+b} \quad (2)$$

We are making the weights that lead to the hidden representation $[H]_{i,j}$ independent of the location of the pixel (i,j) .

This assumption is computationally convenient and makes sense if the patterns that can determine the class of an image can be located anywhere in the image.

From fully connected layers to convolutions

To make the problem even more manageable, we only consider that the hidden unit $[H]_{i,j}$ is influenced by pixels in the vicinity of (i,j) :

$$[H]_{i,j} = u + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} [V]_{a,b} [X]_{i+a,j+b} \quad (3)$$

where Δ is typically smaller than 10.

This means that we should not look very far away from pixel (i,j) to extract relevant information to identify the patterns of that pixel.

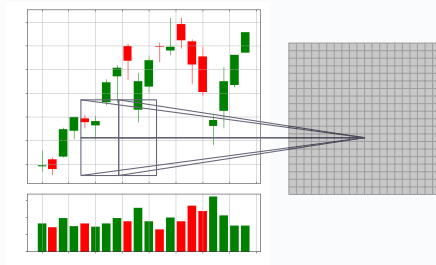
The above expression is what we call a **convolutional layer**.

Convolutional Neural Networks (CNNs) are a special type of neural network that contains convolutional layers, among others, where $[V]_{a,b}$ represents the weights to learn (also called convolutional kernels, or filters).

While the above computations restrict the degree of connections between pixels, the idea is that successive convolutional layers should represent larger and more complex aspects of an image.

Convolutional neural network

Hidden unit, $[H]$



From fully connected layers to convolutions

So far we ignored that images consist of three channels: red, green, and blue. Images are third-order tensors.

- Indeed, modern image files may feature a fourth channel for transparency

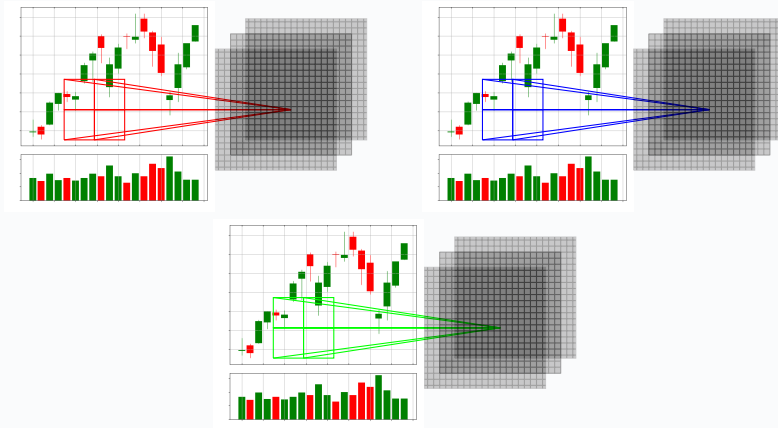
We then need a convolutional filter to have two additional dimensions, $[V]_{a,b,c,d}$, considering that the hidden representation can also have several channels:

$$[H]_{i,j,d} = u_d + \sum_{a=-\Delta}^{\Delta} \sum_{b=-\Delta}^{\Delta} \sum_c [V]_{a,b,c,d} [X]_{i+a,j+b,c} \quad (4)$$

where c indexes the input channels and d indexes the output channels.

While we have so far omitted this, the hidden representation H is usually obtained after the application of an activation function $\phi(\cdot)$ to the outcome of the convolution.

Convolutional neural network



Convolutions for images

Let's see how convolutions work with simple two-dimensional data and hidden representations with a single channel.

INPUT				KERNEL			OUTPUT	
1	2	4		1	2	=	?	?
3	7	2	*	0	1		?	?
2	0	2						

The input tensor has a shape 3×3 , the kernel has a shape or *kernel window* of 2×2 .

We can only properly compute the convolution in locations where the kernel fits wholly in the image so that the output size is

$$(input\ height - kernel\ height + 1) \times (input\ width - kernel\ width + 1) \quad (5)$$

Convolutions for images

Let's see how convolutions work with simple two-dimensional data and hidden representations with a single channel.

$$\begin{array}{c} \text{INPUT} \\ \begin{array}{|c|c|c|} \hline 1 & 2 & 4 \\ \hline 3 & 7 & 2 \\ \hline 2 & 0 & 2 \\ \hline \end{array} \end{array} * \begin{array}{c} \text{KERNEL} \\ \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 0 & 1 \\ \hline \end{array} \end{array} = \begin{array}{c} \text{OUTPUT} \\ \begin{array}{|c|c|} \hline 12 & ? \\ \hline ? & ? \\ \hline \end{array} \end{array}$$

To perform the convolution in an image, we begin with the window positioned at the upper-left corner of the input tensor and slide it both from left to right and top to bottom.

$$\begin{array}{c} \text{INPUT} \\ \begin{array}{|c|c|c|} \hline 1 & 2 & 4 \\ \hline 3 & 7 & 2 \\ \hline 2 & 0 & 2 \\ \hline \end{array} \end{array} * \begin{array}{c} \text{KERNEL} \\ \begin{array}{|c|c|} \hline 1 & 2 \\ \hline 0 & 1 \\ \hline \end{array} \end{array} = \begin{array}{c} \text{OUTPUT} \\ \begin{array}{|c|c|} \hline 12 & 12 \\ \hline 17 & 13 \\ \hline \end{array} \end{array}$$

Warning: Notice that the operator $*$ here represents some abuse of notation, since it refers to element-by-element multiplication and then addition of all terms.

Padding and stride

Convolutional kernels must “fit” into the image, which may lead successive convolutions to “ignore” the boundaries of images.

Several tools allow us to control the size of the output layer and override this problem.

Padding: Consists of adding extra pixels with a value of zero around the boundary of the input image.

INPUT						KERNEL			OUTPUT			
0	0	0	0	0					1	2	4	0
0	1	2	4	0					5	12	12	4
0	3	7	2	0	*	1	2	=	8	17	13	2
0	2	0	2	0		0	1		4	2	4	2
0	0	0	0	0								

With padding, the output size of a convolutional layer is given by:

$$(input\ height - kernel\ height + rows\ of\ padding + 1) \times (input\ width - kernel\ width + columns\ of\ padding + 1)$$

Padding and stride

Stride: Instead of covering the whole input with the kernel in each convolution, we move more than one element at a time.

We may be interested in doing this to reduce computational costs or to downsample.

Above, we used a stride of 1. If we use a stride of 3 horizontally and 2 vertically we get the following:

INPUT						KERNEL			OUTPUT	
0	0	0	0	0						
0	1	2	4	0		1	2		1	0
0	3	7	2	0	*	0	1	=	8	2
0	2	0	2	0						
0	0	0	0	0						

With stride, the output size of a convolutional layer is given by ($\lfloor \cdot \rfloor$ indicates the “floor” operator):

$$\begin{aligned} & \lfloor (input\ height - kernel\ height + rows\ of\ padding + rows\ of\ stride) / rows\ of\ stride \rfloor \\ & \times \\ & \lfloor (input\ width - kernel\ width + columns\ of\ padding + columns\ of\ stride) / columns\ of\ stride \rfloor \end{aligned}$$

Padding and stride

Padding can increase the height and width of the output layer, which is useful to avoid undesirable shrinkage.

Padding also ensures that all pixels are used with the same frequency.

Typically, we opt for symmetric padding on both sides of the input height and width, so that we can refer to padding with a single parameter $p = \text{rows of padding} = \text{columns of padding}$. By default, $p = 0$.

Similarly, when horizontal and vertical strides match, we simply talk about stride s , where by default $s = 1$.

If we set $\text{rows of padding} = \text{kernel height} - 1$, $\text{columns of padding} = \text{kernel width} - 1$, and input's height and width are divisible by the strides on height and width, then the output shape will be

$$(\text{input height} / \text{rows of stride}) \times (\text{input width} / \text{columns of stride})$$

Summary of Lesson 1

In Lesson 1, we have looked at:

- ▶ The concept of convolutional layer in association with image-recognition using neural networks.
- ▶ Padding and stride operations to preserve the dimensionality of input images.

⇒ **References for this lesson:**

Chapter 7 in [Zhang, Aston, et al. "Dive into deep learning." \(2022\).](#)

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will describe the basics of image processing and implement the convolution operator in Python for candlestick charts.